

The Faces of beam-lisp

The pane, the page, and the one write path both of them use

2026-09-18 · verified by driving, not by reading

1 The claim, and how it was tested

beam-lisp has three faces onto the same facts:

- *the pane* — `bl daemon status`, and the same model as a page (`GET /`) and as JSON (`GET /schedules`), read from the schedule store, never by asking the process that owns it;
- *the page* — `examples/hotel/desk.bl`, the hotel's operations rendered with `loom`, whose controls act through `proc/op!` **by name**;
- *the write path* — `proc/op!` (`pause` · `resume` · `run-now`), the one door a face is allowed to press.

The property under test is not “it looks right”. It is: **the same fact appears the same way in every face, and pressing a control in one face moves the fact the others read**. A page that can disagree with the pane makes the pane decoration.

Verification was by **driving**: a real daemon, a real Bandit listener, a real browser, and the store read independently with `curl` after every click. Reading the source would have found none of the four defects in §5.

2 The pane

```
./bin/bl daemon start          # detached; the UI claims a named port
./bin/bl daemon status        # the terminal face of the same model
# → port ui = http://beam-lisp.test/ → 53155
```

The terminal face and the page agree cell for cell, because they are two renderings of one projection (`vm.inspect/model`):

schedules	2 declared				
schedule	beam-lisp@574618/cache-prune	paused	0 runs	in 17798s	
schedule	beam-lisp@574618/index-refresh	active	0 runs	in 1719s	
ticker	bl-sched/beam-lisp@574618	alive	pid #PID<0.5401.0>		

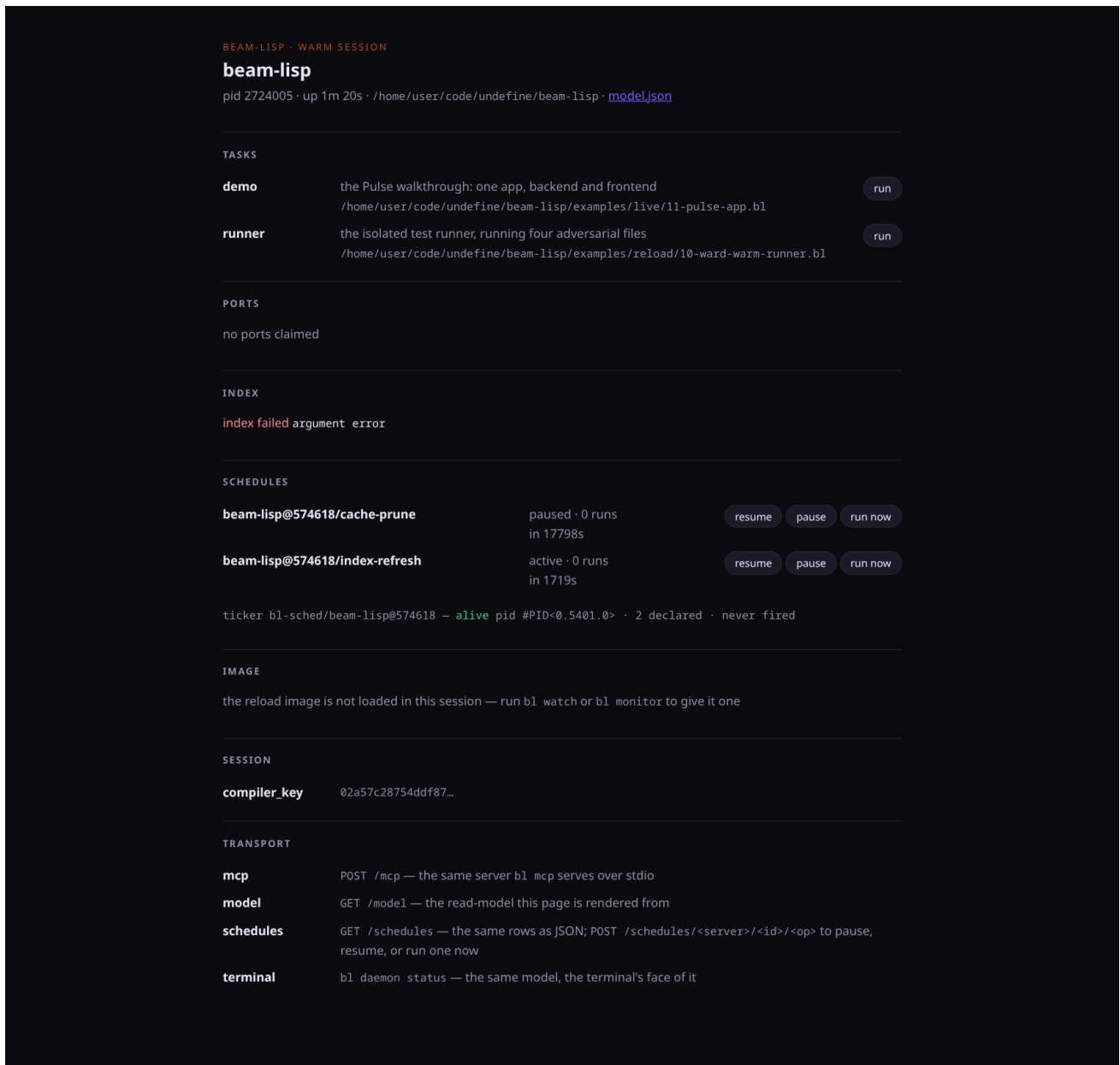


Figure 1: The pane as it stands: the schedules with their controls, the **ticker line** (the one live thing in a store-only view, drawn as a monitor's fact), the reload image's state, and the transport — every route this page answers to. *cache-prune* reads *paused* here because *\$4* paused it.

The first capture of this page was **blank**, which is worth recording: it was a capture that happened before the navigation settled, not a broken page — the body was 5,419 bytes of real content at the same moment. A screenshot is only evidence when the reading agrees with it, which is why every figure here is paired with a command's output.

3 The page

`./bin/bl serve examples/hotel/desk.bl` # http://127.0.0.1:4048

the hotel · front desk

The desk's rooms, the night ledger, and the wheel's schedules — read off the hotel in examples/hotel/hotel.bl, never copied from it.

the desk

room 101
guest: Ada

occupied

room 102
no guest

vacant

room 103
guest: Grace

occupied

the night ledger

room 101	174
room 102	210
total	384

the wheel

The hotel's clock is a value that starts at midnight, so nothing is due until the walkthrough moves it — or you click run now.

housekeeping / audit

active

running

runs 0 · fails 0 · next in 10800s

pause

run now

housekeeping / rounds

active

running

runs 0 · fails 0 · next in 150ms

pause

run now

pause · resume · run-now go through proc/op! by name. The page holds nothing between requests: reload it and you see whatever the hotel now holds.

Figure 2: The hotel's front desk: the desk's rooms (from the running desk process), the night ledger (from the table the heir holds), and the wheel (from the store) — with a control per schedule.

The page holds nothing between requests, and its own footer says so. That is what makes a control a **name** rather than a handle: the button carries `?server=housekeeping&id=audit&op=pause`, and the operation is the same verb the terminal and the pane call.

4 The write path, driven

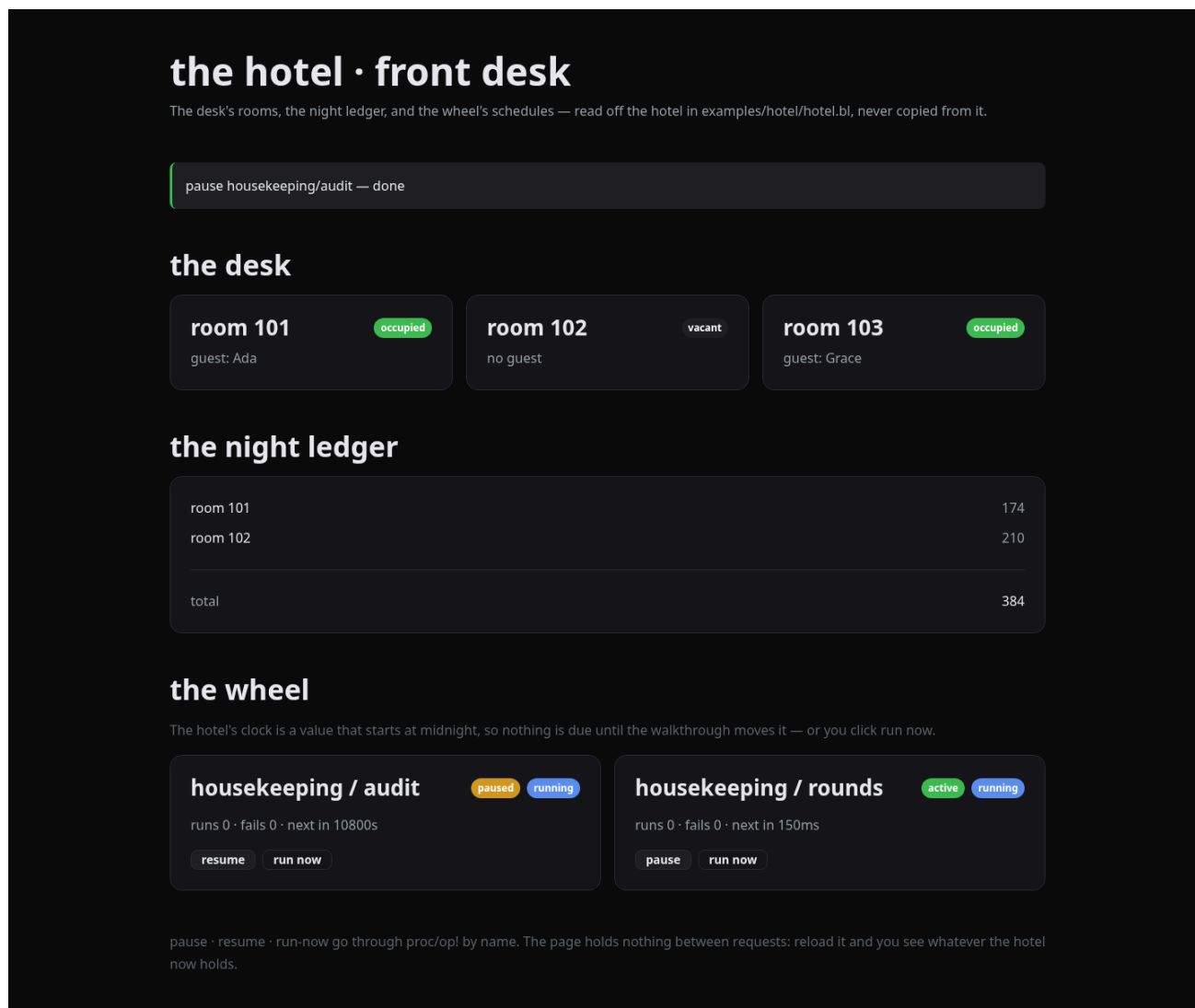


Figure 3: After clicking *pause* on *housekeeping/audit*: the banner reports the operation by name, the badge reads *paused*, and — the part worth noticing — the control set *CHANGED*: *resume* appeared beside *run now*. The other schedule is *untouched*.

Read independently, in the same second:

```
$ curl -s 'localhost:4048/?server=housekeeping&id=audit&op=pause'
... housekeeping / audit  paused  running  runs 0 ...
$ curl -s localhost:4048/ | grep -o 'paused'
paused
```

The state survives the request — the second read is a fresh page, not a one-shot message — because *proc/op!* writes the *store* and the page renders the store.

5 The same property on the pane's own face

The pane's controls are not links but *fetch* calls carrying the session token, so they were driven by invoking the page's own handler and comparing the result with the store, read separately:

```
in the page (JS):  await schedOp('beam-lisp@574618','cache-prune','pause')
                   → "pause beam-lisp@574618/cache-prune ok"
```

```
the store:        GET /schedules → [{"id":"cache-prune", ... "state":"paused"}]
the terminal:      bl daemon status → beam-lisp@574618/cache-prune  paused  0 runs
```

Three faces, one fact. Note that the *synthetic* click on those buttons did not fire the handler in the harness's browser (its observer sees no interactive nodes on these pages), so the JS path was invoked

directly. That is a limitation of how this document was produced, not a claim about the page: the handler it invoked is the handler the button carries.

6 What driving found (all four were invisible in the source)

(1) The buttons did not work at all, and said the wrong thing.

The router matched method+path, so the parameterised route compared **what the browser sent** with **what the store holds**: the browser sends `beam-lisp%40574618` where the store holds `beam-lisp@574618`. The failure read **no scheduler is running for this server** — while the scheduler was running. A request hand-written with a raw `@` worked, which is exactly why this survived reading. Fixed by percent-decoding each path segment (`priv/std/vm/http.bl::pct-decode`), with a comment recording the measurement. Two other defects were visible in the same capture, and are fixed with it:

(2) **three unexplained grey bars**. The dashboard has three `<pre id="...-out">` slots for button output. Before anything is pressed they rendered as three bordered rectangles promising an answer. `pre:empty{display:none}`.

(3) **the schedule key wrapped into its own stats**. `td.name{width:9rem}` is right for a task name and wrong for `beam-lisp@574618/cache-prune`, which wrapped to three lines and collided with the column beside it. A `td.key` class that never wraps.

(4) **a page whose whole body rendered nothing**. The first hotel draft returned 200 with an empty body: `loom.token/with-sheet` takes a **thunk**, and a tree raised `ArgumentError: a vector called with 0 arguments inside the render`. It looked correct in the source. (FUP-104 recorded the fix; this document is the first place it was driven.)

7 Also observed, not fixed

The index row can read **index failed argument error and stay there**. Root cause, found with `bl` search:

```
error: could not read file "tmp/rename-proto/bad/app/src/app/unreadable.bl":
      permission denied
```

One unreadable file — a test fixture — fails the whole index walk, and the pane reports the exception's message with no path and no context. Two fixes belong here and neither is in this change: the walk should skip an unreadable file **and report it**, and the error row should carry the path it choked on.

One transient red tree. Six `bl` processes compiling at once left the shared AOT cache (`~/.cache/beam_lisp/aot`) corrupt, and every command then died with a compiler crash pointing at `priv/boot/reader-node.bl` — a file nobody had touched. Clearing the cache fixed it instantly. Filed as BUG-083: the write is not atomic and a bad entry is not treated as a miss.

8 Reproduction

```
./bin/bl daemon start && ./bin/bl daemon status    # the pane; the ui port is named
./bin/bl serve examples/hotel/desk.bl              # the page on :4048
curl -s localhost:4048/                            # rooms, ledger, wheel
curl -s 'localhost:4048/?server=housekeeping&id=audit&op=pause'
typst compile reports/ui/ui.typ reports/ui/ui.pdf  # this document
```

Screenshots: `shots/pane-01-full.png` (before the layout fixes), `shots/pane-02-fixed.png`, `shots/hotel-01.png`, `shots/hotel-02-paused.png`.